	Pimpri Chinchwad Education Trust’s **Pimpri Chinchwad College of Engineering** An Autonomous Institute (Permanently affiliated to Savitribai Phule Pune University)	**Semester VII**

Mini Project

Name: Tanushka Patil, Manali Thorat
PRN: 123B1B036, 123B1B037
Class: B.Tech A
Subject: Distributed Systems (DS)

Title: Snip: A Distributed URL Shortener

1. Student Details

Sr. No.	Name	PRN	Division
1	Tanushka Patil	123B1B036	A
2	Manali Thorat	123B1B037	A

2. Project Title and Description

Title: Snip: A Distributed URL Shortener built with Microservices, Docker and Kubernetes

Snip turns long URLs into short links (for example `/k8s-docs`) and redirects visitors to the original address. It is built as a set of independent microservices rather than a single application, so that it can show the core ideas of distributed systems in a working product: horizontal scaling, load balancing, caching, synchronous and asynchronous inter-service communication, and fault tolerance. Users can create links with an optional custom alias and expiry time, view recent links, see per-link click analytics, and watch requests being spread across service replicas from a live web dashboard. The same system runs on Docker Compose and on Kubernetes, where it autoscales under load.

3. Git Repository

https://github.com/tanushkapatil/URL_Shortner

4. Implementation Details

4.1 Architecture

All traffic enters through an Nginx gateway, which serves the web UI and routes each API path to the service that owns it, round-robining across that service's replicas. The write path (shortener) and the read path (redirect) are separate services, so the much busier redirect path can scale on its own. Click events are published to a Redis Stream and consumed by the analytics service, so a redirect never waits for analytics.

Component	Role	Replicas
Gateway (Nginx)	Single entry point: serves the UI, API routing, load balancing, retries a failed replica	1 (Compose) / 2 (K8s)
Shortener (Node.js / Express)	Create, list and delete links; custom aliases; expiry; distributed rate limiting. Owns the *urls* table	2
Redirect (Node.js / Express)	Hot path: resolves a code and returns a 302 redirect. Stateless, Redis-cached	3 to 10 (HPA)
Analytics (Node.js / Express)	Consumes click events from the queue and serves statistics. Owns the *clicks* table	2
PostgreSQL 16	Persistent storage; each service owns its own tables	1
Redis 7	Read cache, click-event stream (message queue), shared rate-limit counters	1

4.2 Request Flows

- **Create link:** `POST /api/shorten` → gateway → any shortener replica → atomic Postgres sequence value → encoded as a 7-character base62 code → stored → 201 Created.
- **Follow link:** `GET /:code` → gateway → any redirect replica → Redis cache lookup (on a miss, a REST call to the shortener, then cached) → 302 redirect. In the background a click event is added to the Redis Stream and processed by exactly one analytics replica.

4.3 Distributed-Systems Concepts Implemented

Concept	How it is implemented
Microservices	Three independently deployable services, each owning its own data
Synchronous REST	redirect → shortener (link lookup on cache miss); shortener → analytics (click counts)
Asynchronous messaging	Click events flow through a Redis Stream so redirects never block on analytics
Load balancing	Nginx round-robin across replica IPs (Docker DNS) or a Kubernetes Service
Horizontal scaling / autoscaling	`docker compose --scale`; Kubernetes HorizontalPodAutoscaler on CPU
Competing consumers	Analytics replicas share one Redis consumer group; each event is processed once
Distributed ID generation	Postgres sequence + bijective base62 scramble: unique, non-guessable codes, no collisions
Caching	Redis read-through cache with TTL, negative caching and invalidation on create/delete
Distributed rate limiting	Counters in Redis shared by all shortener replicas (30 links/min per client)
Fault tolerance	Gateway retries another replica; services degrade gracefully when a dependency is down
At-least-once delivery + idempotency	Ack after DB write; un-acked events reclaimed with XAUTOCLAIM; duplicates rejected by a UNIQUE constraint
Health checks / self-healing	/live and /health endpoints; Docker healthchecks; Kubernetes startup, liveness and readiness probes
Graceful shutdown	SIGTERM handling and preStop hooks give zero-downtime rolling updates

4.4 Technologies and Tools Used

Area	Technologies / Tools
Backend services	Node.js 20, Express 4, node-postgres (pg), ioredis
API gateway / load balancer	Nginx 1.27
Database	PostgreSQL 16
Cache and message queue	Redis 7 (key-value cache, Redis Streams with consumer groups)
Frontend	HTML, CSS, vanilla JavaScript dashboard served by the gateway
Containerisation	Docker, Docker Compose (multi-replica deployment)
Orchestration	Kubernetes (Deployments, Services, ConfigMap, Secret, PVC, HPA, probes), Kustomize, metrics-server
Testing / public access	Custom zero-dependency Node.js load tester; Cloudflare Tunnel for public demo URL
Version control	Git and GitHub

4.5 API Summary

Method	Path	Description
POST	/api/shorten	Create a short link (url, optional alias, optional expiry)
GET	/api/urls	Recent links with click counts
DELETE	/api/urls/:code	Delete a link and invalidate its cache entry
GET	/:code	302 redirect (404 unknown, 410 expired); X-Served-By and X-Cache headers
GET	/api/stats, /api/stats/:code	Cluster overview and per-link statistics
GET	/health/{service}	Health of one replica and its dependencies

5. Screenshots of the Working Application

The screenshots below are taken from a screen recording of the application deployed on Kubernetes (Docker Desktop cluster, gateway exposed at `localhost:30080`).

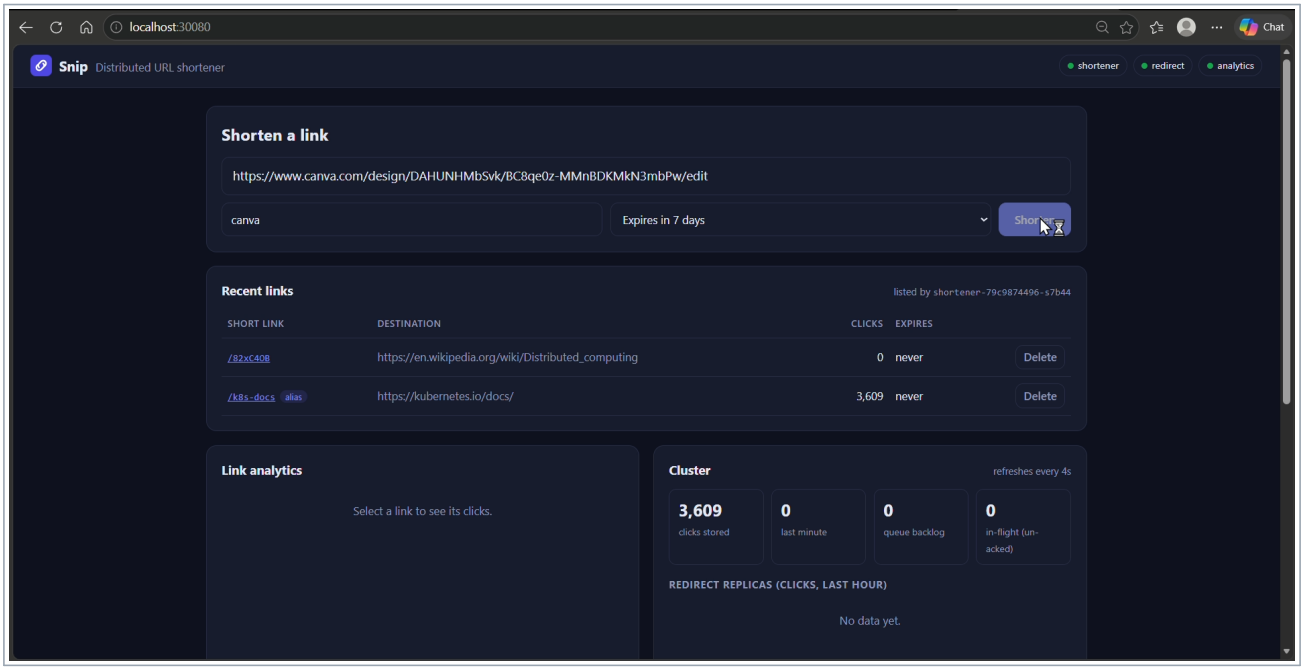


Figure 1: Snip web dashboard. A long Canva URL is entered with the custom alias *canva* and a 7-day expiry. The status pills (top right) show the shortener, redirect and analytics services healthy.

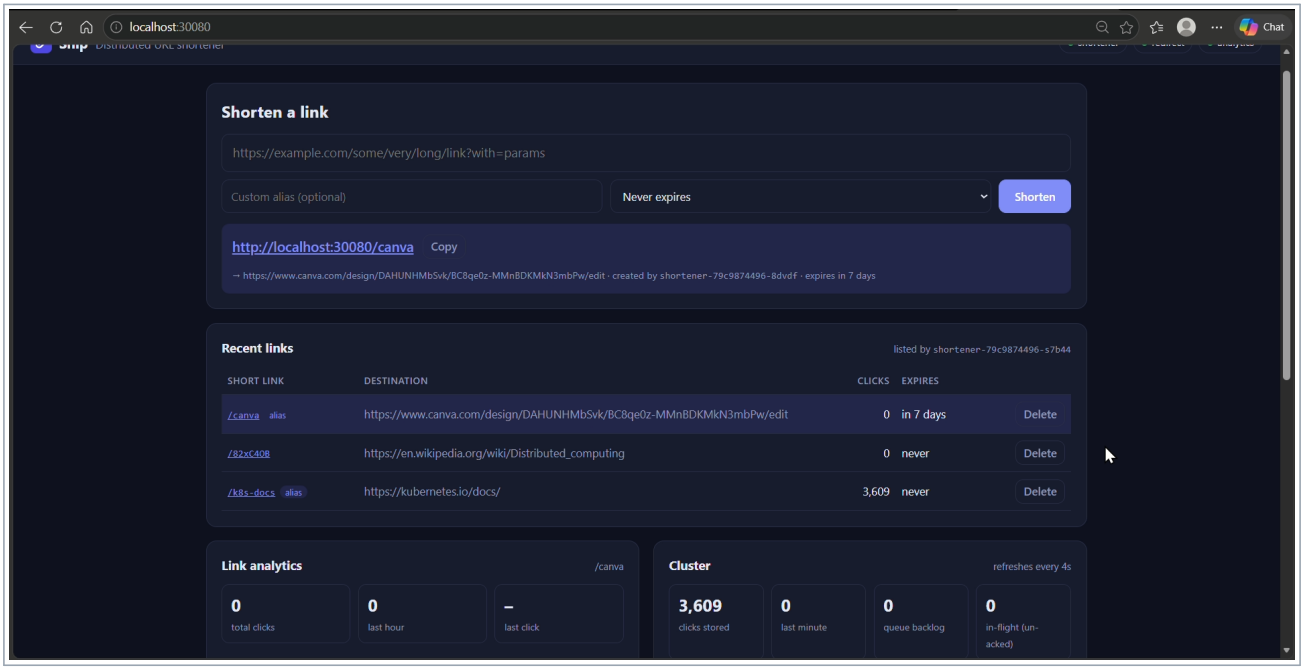


Figure 2: The short link `http://localhost:30080/canva` is created and appears in the Recent links table, which also lists click counts and expiry for each link.

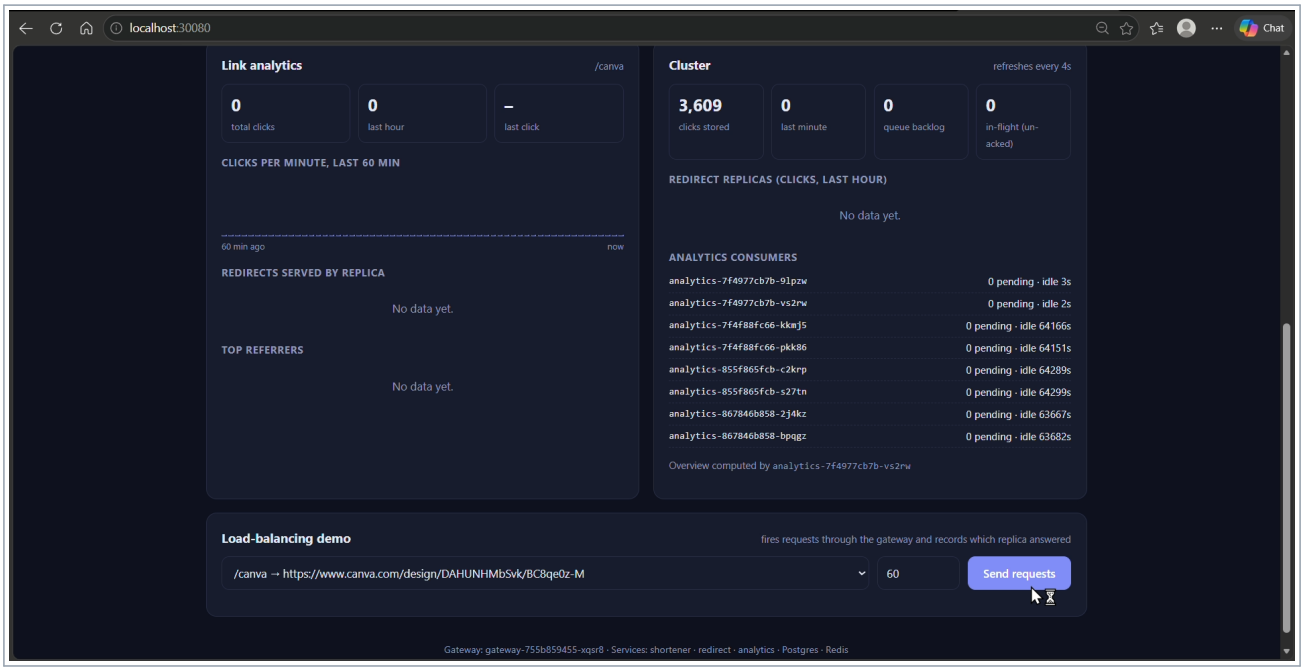


Figure 3: Link analytics and Cluster panels. The cluster overview shows total clicks stored, queue backlog and the analytics replicas consuming click events from the Redis Stream consumer group.

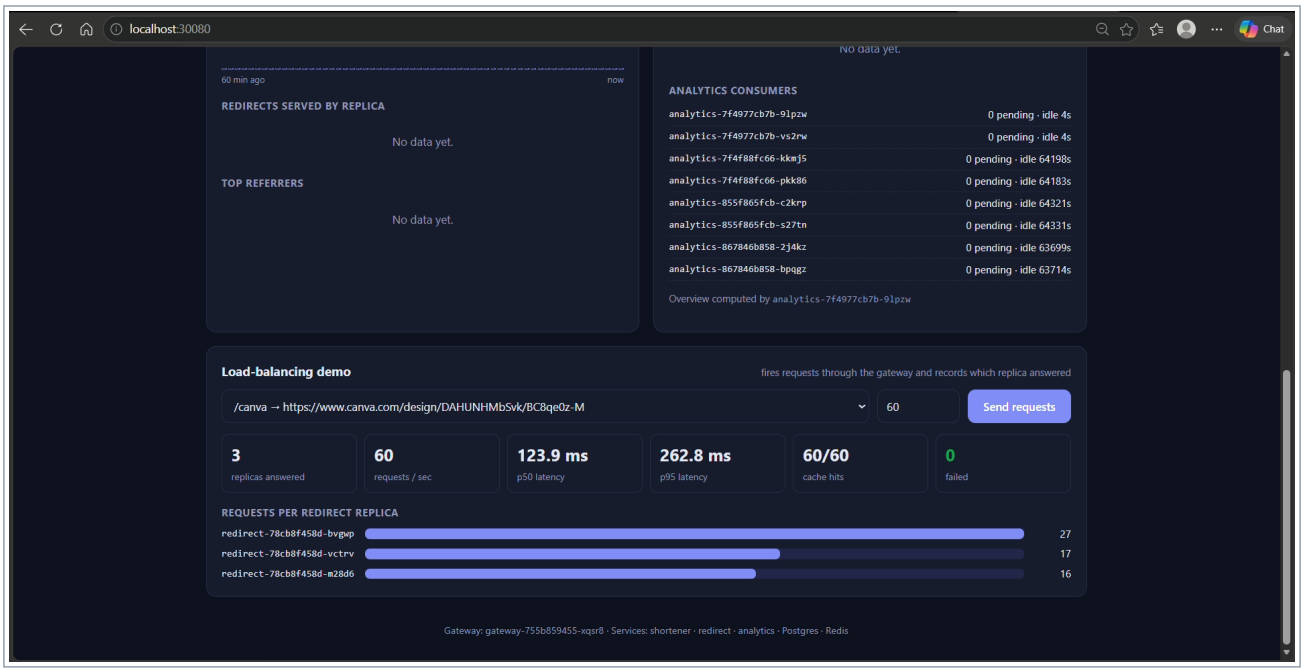


Figure 4: Load-balancing demo. 60 requests sent through the gateway were answered by 3 redirect pods (27 / 17 / 16), with 60/60 Redis cache hits, p50 latency 123.9 ms and 0 failures.

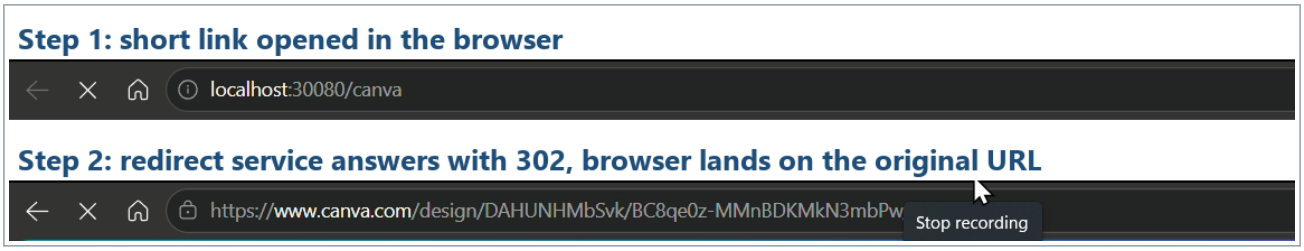


Figure 5: Opening the short link in the browser. The redirect service returns a 302 and the browser lands on the original long URL.

6. Conclusion

We built and deployed a working distributed URL shortener made of three microservices behind an Nginx gateway, backed by PostgreSQL and Redis, and runnable on both Docker Compose and Kubernetes. The project shows the main distributed-systems concepts in practice: traffic is load balanced evenly across replicas, the redirect service scales out independently (Kubernetes autoscaled it from 3 to 10 pods under load, serving over 30,000 requests with zero errors), and the Redis cache and asynchronous click queue keep redirects fast. Fault-tolerance tests confirmed that killing a replica, stopping the analytics service, or stopping Redis does not break redirects, and that queued clicks are processed without loss once a service recovers. Through this project we gained hands-on experience with service decomposition, inter-service communication, caching, message queues, container orchestration and designing for failure.